TAR System Description Paper Template

Author1, Author2, Author3

University of Zagreb, Faculty of Electrical Engineering and Computing
Unska 3, 10000 Zagreb, Croatia
autor1@xxx.hr, {autor2,autor3}@zz.com

Abstract

This document provides the instructions on formatting the TAR system description paper in LaTeX. This is where you write the abstract (i.e., summary) of the work you carried out within the project. The abstract is a paragraph of text ranging between 70 and 150 words.

1. Introduction

Text for the introduction goes here.

2. Related work

Text for the related work goes here.

3. Methodology

In this section, we describe the established framework for evaluating the efficacy and robustness of pairing Negative Preference Optimisation (NPO) with Low-Rank Adaptation (LoRA) for large language model unlearning. Additionally, we define the adversarial jailbreak attack evaluation framework. We detail the datasets, models, and evaluation metrics employed to assess the performance of such an approach.

3.1. Tasks and dataset

The main dataset used in experiments is part of the *LUME: LLM Unlearning with Multitask Evaluations* benchmark. The dataset consists of documents that are categorised into three scenarios:

(I) **Synthetic creative documents** This scenario consists of synthetic fictional stories of different genres.

(II) **PII-containing synthetic biographies** This scenario consists of synthetic biographies of public figures that contain personally identifiable information (PII) such as names, home addresses, and Social Security numbers.

(III) **Real documents from pretraining corpus** This scenario consists of real documents that were part of the pretraining corpus of the language model.

Each of the subtasks of the challenge contains two main data splits: *Forget* and *Retain*. The *Forget* split contains documents that the model should unlearn, while the *Retain* split contains documents that should be retained in the model's memory. The number of examples can be found in Table 1.

Table 1: Distribution of documents in the dataset.

Document	Forget	Retain	Total
Synthetic Creative Documents	166	206	372
Synthetic Biographies	642	612	1254
Real Documents	304	318	622
Total	1112	1136	2248

3.2. Model

Our primary experiments are conducted using the *OLMo-7B-0724-Instruct-hf* model, which has been explicitly fine-tuned to memorise the documents from the LUME benchmark dataset.

Unlearning the above-mentioned model is achieved through Negative Preference Optimisation (NPO). As this method is computationally expensive, we apply it in combination with Low-Rank Adaptation (LoRA) to reduce the number of trainable parameters, and thus make the unlearning process more efficient. All experiments were conducted on a single NVIDIA A100 40GB GPU.

Given a prompt x, the training objective of our approach is to minimise the likelihood of generating forgotten documents y_f, relative to a fixed reference model π_{ref}. Formally, the training objective can be expressed as follows:

$$\mathcal{L}_{NPO}(\theta) = \frac{2}{\beta} \log \left(1 + \left(\frac{\pi_\theta(y_f|x)}{\pi_{ref}(y_f|x)} \right)^\beta \right),$$

where π_θ is the model being trained, π_{ref} is the reference model, and β is a hyperparameter that controls the strength of the preference. Furthermore, to preserve the model's general capabilities, we include an additional cross-entropy loss term that encourages the model to retain the knowledge of the retained documents y_r. The final loss function is:

$$\mathcal{L}(\theta) = \mathcal{L}_{NPO}(\theta) + \alpha \cdot \mathcal{L}_{CE}(\theta),$$

where $\mathcal{L}_{CE}(\theta)$ is the cross-entropy loss computed on the retained documents, and α is a hyperparameter that aids in balancing unlearning and knowledge retention.

To further strengthen our jailbreak evaluation comparisons, we implemented Activation Steering method proposed in (Stolfo et al., 2024). Activation steering works by identifying a desired direction in the latent space of the model and suppressing or enhancing it at inference time. Given a set of N pairs (x_f, x_r), where x_f represents a forget-set prompt and x_r represents a retain-set prompt, we extract hidden-state activations h_f and h_r at decoder layer

$\ell = 20$ by mean-pooling over the token dimension. The steering vector is then computed as the mean difference:

$$\mathbf{v} = \frac{1}{N} \sum_{i=1}^{N} \left(h_f^{(i)} - h_r^{(i)} \right), \quad \hat{\mathbf{v}} = \frac{\mathbf{v}}{\|\mathbf{v}\|},$$

where $\hat{\mathbf{v}} \in \mathbb{R}^d$ is the unit-normalised steering vector. At inference time, a forward hook is applied on layer ℓ that modifies the forward pass activations at every token position:

$$h_\ell^{\text{out}} \leftarrow h_\ell^{\text{out}} + \alpha \cdot \hat{\mathbf{v}},$$

where $\alpha < 0$ is the steering strength. Setting α negative subtracts the memorisation direction, thereby suppressing information recall without modifying the model weights. In our experiments, we use $N = 64$ contrast pairs and $\alpha = -5.0$, selected by sweeping over $\alpha \in \{-2.0, -5.0, -8.0, -12.0, -20.0\}$ to balance unlearning effectiveness and output quality. The steering vector is extracted from the baseline model fine-tuned to memorise the dataset, prior to unlearning, and remains fixed throughout training and evaluation.

3.3. Adversarial Red-Teaming

To evaluate the robustness of NPO-LoRA against adversarial attacks, we implemented an adversarial red-teaming procedure. We utilized *gpt-4o-mini* to rewrite the queries from the *Forget* split into seven different adversarial prompts:

- **Paraphrase** - altering the wording of the original query while preserving its meaning.
- **Prefix Injection** - prepends bypass instructions to the original query.
- **Roleplay** - frames the query as a roleplay scenario.
- **Translation** - translates the original query to another language.
- **Code Context** - embeds the original query within a code snippet.
- **Neighbour** - targets adjacent documents without referencing the original query.
- **Logical Chain** - multi-step reasoning that leads to the original query.

3.4. Evaluation Metrics

Our evaluation framework relies on the competition metrics, which aggregate three core dimensions via an arithmetic mean to generate a final submission score. First, task-specific rates are computed across the *Forget* and *Retain* splits. A ROUGE-L regurgitation score is used for sentence completion tasks, while Exact Match (EM) is used for question-answering tasks. Importantly, the forget split metrics are inverted (1 − value). In total, 12 scores across the three tasks are then aggregated via the harmonic mean into a single score that reflects the overall performance of the unlearning approach. Second, a Membership Inference Attack (MIA) score is computed over member and non-member documents to assess the model's vulnerability to privacy attacks. Third, general capabilities are evaluated using the MMLU benchmark, and the performance is measured as the average accuracy across all tasks.

4. Results

We evaluate the performance of NPO-LoRA by tracking metrics across varying LoRA adapter capacities and NPO hyperparameters. The results are presented in Table 2, which is split into three column groups: hyperparameters, base metrics, and aggregate scores. The *Hyperparameters* column group includes the LoRA rank (r), LoRA scaling factor (α_{LoRA}), NPO preference strength (β_{NPO}), NPO knowledge retention weight (α_{NPO}), learning rate (LR), and the number of training epochs (Ep.). The *Base Metrics* reports the performance on the *Forget* and *Retain* splits. The arrows (↓, ↑) denote the desired direction of improvement. Finally, the *Aggregate* section reports the Membership Inference Attack (MIA) score and the overall final score (Agg).

The **Baseline** model represents the original model without any unlearning applied. This model achieves a maximum score of 1.000 across all base metrics and a MIA score of 0.0, resulting in the lowest-overall aggregate score of 0.168. This score serves as a reference point for evaluating the effectiveness of different NPO-LoRA configurations.

Configurations with low to mid-capacity LoRA adapters ($r = 8, 16$) showed limited performance, with the best configuration (Run 3) achieving an aggregate score of 0.230. Scaling up the LoRA adapter capacity to $r = 32$ (Runs 5-8) allowed for significant improvements in the aggregate score, with the best configuration (Run 6) achieving an aggregate score of 0.318. This configuration also achieved the lowest F-Reg (0.720) and F-Know (0.553) scores. Run 7, which had a slightly higher NPO knowledge retention weight, achieved a slightly lower aggregate score of 0.301 but maintained higher R-Reg (0.968) and R-Know (0.929) scores, at the cost of slightly higher F-Reg (0.779) and F-Know (0.583) scores.

Across all evaluated configurations, including the baseline, the MIA score remained static (0.0).

5. Discussion

Text for the discussion goes here.

6. Conclusion

Conclusion is the last enumerated section of the paper. It should not exceed half of a column and is typically split into 2–3 paragraphs. No new information should be presented in the conclusion; this section only summarizes and concludes the paper.

Acknowledgements

We would like to thank the University Computing Centre (SRCE) in Zagreb for providing the computational resources on their Advanced Computing platform used in this work.

Table 2: Hyperparameter ablation study including the **Baseline** model.

	Hyperparameters						Base Metrics				Aggregate	
Run	r	α_{LoRA}	β_{NPO}	α_{NPO}	LR	Ep.	F-Reg ↓	F-Know ↓	R-Reg ↑	R-Know ↑	MIA ↑	Agg ↑
Baseline	-	-	-	-	-	-	1.000	1.000	1.000	1.000	0.0	0.168
Low / Mid Capacity Adapters ($r = 8, 16$)												
1	8	16	0.05	1.0	1e-5	2	0.953	0.960	0.960	0.969	0.0	0.169
2	16	32	0.10	1.5	3e-5	1	0.933	0.809	0.948	0.872	0.0	0.223
3	16	32	0.15	1.2	5e-5	2	0.925	0.819	0.982	0.962	0.0	0.230
4	16	32	1.00	1.0	5e-5	2	0.983	0.931	0.992	0.991	0.0	0.185
High Capacity Adapters ($r = 32$)												
5	32	64	0.08	0.5	3e-5	2	0.852	0.645	0.962	0.933	0.0	0.270
6	**32**	**64**	**0.10**	**0.1**	**1e-4**	**3**	**0.720**	**0.553**	0.885	0.848	0.0	**0.318**
7	**32**	**64**	**0.10**	**0.2**	**1e-4**	**3**	0.779	0.583	**0.968**	**0.929**	0.0	0.301
8	32	64	0.30	1.0	1e-4	4	0.913	0.797	0.989	0.986	0.0	0.232

Table 3: Attack Success Rate (ASR) summary across models.

	ASR by Model		
Attack	**Baseline**	**Run 6**	**Steering**
Prefix Injection	85.4%	78.2%	0.2%
Roleplay	78.5%	68.1%	0.0%
Paraphrase	73.7%	61.2%	1.8%
Translation	68.0%	60.0%	5.7%
Logical Chain	59.2%	48.9%	0.0%
Neighbour	45.0%	38.7%	0.8%
Code Context	1.1%	1.1%	0.0%

References

Alessandro Stolfo, Vidhisha Balachandran, Safoora Yousefi, Eric Horvitz, and Besmira Nushi. 2024. Improving instruction-following in language models through activation steering. *arXiv preprint arXiv:2410.12877*.